

JAVA PROGRAMMING LABORATORY

Experiment No: Week 11

Date: 16/09/26

Title: Exception Handling – Mobile Number Validation

1. Objective / Aim

To develop a Java program that validates a mobile number using exception handling and a user-defined exception. The program accepts a mobile number, checks whether it contains exactly 10 digits, and handles invalid input using a custom exception.

2. System Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Minimum 10 GB free storage
- Keyboard and monitor

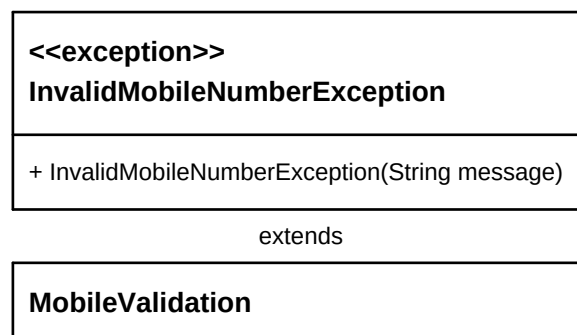
Software Requirements

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK): 17 or later
- IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
- Command Prompt / Terminal

3. Algorithm / Logic Description

1. Start the program.
2. Import the Scanner class.
3. Create a custom exception named InvalidMobileNumberException by extending Exception.
4. Create a Scanner object to read the mobile number as a String.
5. Use matches("[0-9]{10}") to check whether the mobile number contains exactly 10 digits.
6. If the number is invalid, throw the custom exception.
7. Catch the exception using a catch block and display the error message.
8. If the number is valid, display "Mobile Number is Valid."
9. Close the Scanner object.
10. Stop the program.

4. UML Class Diagram



```
+ main(String[] args): void
```

5. Source Code

```
import java.util.Scanner;

// Custom exception class for invalid mobile number
class InvalidMobileNumberException extends Exception {

    // Constructor to display the error message
    public InvalidMobileNumberException(String message) {
        super(message);
    }
}

// Main class
public class MobileValidation {

    public static void main(String[] args) {

        // Create Scanner object to take input
        Scanner sc = new Scanner(System.in);

        // Ask the user to enter mobile number
        System.out.print("Enter Mobile Number: ");
        String mobile = sc.nextLine();

        try {

            // Check whether mobile number has exactly 10 digits
            if (!mobile.matches("[0-9]{10}")) {

                // Throw custom exception if number is invalid
                throw new InvalidMobileNumberException(
                    "Invalid Mobile Number! Enter exactly 10 digits."
                );
            }

            // Display message when mobile number is valid
            System.out.println("Mobile Number is Valid.");

        } catch (InvalidMobileNumberException e) {

            // Handle the invalid mobile number exception
            System.out.println("Exception: " + e.getMessage());
        }

        // Close Scanner object
        sc.close();
    }
}
```

6. Compilation and Execution Commands

Step 1: Save the program

Save the source file as **MobileValidation.java**.

Step 2: Compile

```
javac MobileValidation.java
```

If compilation is successful, Java creates the required .class files.

Step 3: Execute

```
java MobileValidation
```

7. Output

Test Case 1 – Valid Mobile Number

```
Enter Mobile Number: 9876543210
```

```
Mobile Number is Valid.
```

Test Case 2 – Invalid Mobile Number

```
Enter Mobile Number: 98765
```

```
Exception: Invalid Mobile Number! Enter exactly 10 digits.
```

8. Result

The Java program was successfully designed to validate a mobile number using exception handling and a user-defined exception. Valid 10-digit numbers are accepted, while invalid numbers are handled with an appropriate exception message.